

To create the model of the user creation:

```
namespace JLoka.ChatService.Models;

public class UserConnection
{
    public string Username { get; set; } = string.Empty;
    public string ChatRoom { get; set; } = string.Empty;
}
```

=====

To create a simple layer for the data service:

```
using JLoka.ChatService.Models;
using System.Collections.Concurrent;

namespace JLoka.ChatService.DataService
{
    public class SharedDb
    {
        private readonly ConcurrentDictionary<string, UserConnection> _connections = new
        ConcurrentDictionary<string, UserConnection>();
        public ConcurrentDictionary<string, UserConnection> connections => _connections;
    }
}
```

=====

To create the chat hub to create, send, and join chat rooms and messages:

```
using Microsoft.AspNetCore.SignalR;
using JLoka.ChatService.Models;
using JLoka.ChatService.DataService;

namespace JLoka.ChatService.Hubs;

public class ChatHub: Hub
{
    private readonly SharedDb _shared;

    public ChatHub(SharedDb shared) => _shared = shared;

    public async Task JoinChat(UserConnection conn)
    {
        await Clients.All.SendAsync("ReceiveMessage", "admin", $"{conn.Username} has
joined");
    }

    public async Task JoinSpecificChatRoom(UserConnection conn)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, conn.ChatRoom);
        _shared.connections[Context.ConnectionId] = conn;
        await Clients.Group(conn.ChatRoom).SendAsync("JoinSpecificChatRoom", "admin",
        $"{conn.Username} has joined {conn.ChatRoom}");
    }
}
```

```

        public async Task SendMessage(string msg)
        {
            if(!_shared.connections.TryGetValue(Context.ConnectionId, out UserConnection
conn))
            {
                await Clients.Group(conn.ChatRoom).SendAsync("ReceiveSpecificMessage",
conn.Username, msg);
            }
        }
    }
}

```

=====

To use the SignalR, Hubs, and CORS with the ASP.Net Core Application:

```

using JLoka.ChatService.DataService;
using JLoka.ChatService.Hubs;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddSignalR();

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

builder.Services.AddCors(opts =>
{
    opts.AddPolicy("reactApp", builder =>
    {
        builder.WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});

builder.Services.AddSingleton<SharedDb>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseCors("reactApp");
app.MapControllers();
app.MapHub<ChatHub>("/Chat");

app.Run();

```

=====